GENERATIVE AI TRAINING

Course Notes Q&A Assistant

System Architecture and Technical Design

A Python CLI that retrieves course evidence from local Chroma and produces cited answers through a model API.

Prepared for Shashank Jha
Version 1.0 | 1 November 2026
Design baseline - application not implemented

01-03 Scope, student workflow and component boundaries

04-06 Knowledge sources, provenance and index lifecycle

07-09 Retrieval, grounded answers, limits and privacy

10-12 Package design, evaluation, decisions and references

1. Purpose, scope and design authority

Build a terminal application that answers student questions using approved course notes. Retrieval finds relevant passages in a locally persisted Chroma collection; a language model receives those passages and the question, then returns a grounded answer with source references or an explanation that evidence is insufficient.

The supplied Course Notes Q&A Assistant Lesson Plan governs learning outcomes and product scope. This architecture defines the proposed engineering baseline; DEVELOPMENT_PLAN.md turns it into ordered implementation work. The editable Markdown is the architecture source; its PDF is a generated reading copy. Update both together when decisions change.

Initial release

- Python command-line application for one local user and one active knowledge-base snapshot.
- Explicit ingestion of allowlisted Markdown, UTF-8 text and text-based PDF notes.
- Local Chroma persistent storage; no separately operated database server.
- One explicitly configured embedding adapter and one generation API adapter.
- Single questions or an interactive question loop; each question is independent.
- Answers with file/section or PDF-page citations, plus supported abstention and controlled errors.
- Retrieval inspection and a small repeatable evaluation set.

Outside the initial release

No web interface, separate backend, user accounts, shared hosting, agents/tools, internet search, autonomous file discovery, conversation memory, fine-tuning, OCR, image understanding, raw spreadsheet analysis, or production multi-user deployment. Documents and links are never executed or fetched automatically.

Required versus proposed

The lesson plan requires the CLI, Python, model API, local persistent Chroma, document preparation, source display, missing-evidence handling and evaluation. Module names, command names, snapshot lifecycle, numeric defaults and acceptance targets in this document are proposed design decisions. They are not claims about implemented behavior or measured quality.

Change control

Record scope/contract changes with the decision, reason, affected requirements, migration impact and verification. Update the Markdown, regenerated PDF and development plan together. A plan checkbox is evidence tracking, not proof that the application already exists.

2. User journey and functional requirements

The student first prepares an index, inspects extracted passages, then asks questions. Asking never silently rebuilds the index. A changed source requires explicit ingestion before new material becomes queryable.

Proposed command interface

Commands below describe the intended interface; they are not available yet.

```
course-qa ingest --manifest configs/sources.yaml
course-qa status
course-qa retrieve "Why blur frames before differencing?"
course-qa ask "Why blur frames before differencing?"
course-qa chat
course-qa evaluate --cases eval/cases.jsonl
```

`chat` is a repeated independent-question loop, not a memory-based chatbot. `:quit`, EOF and Ctrl+C exit cleanly. `retrieve` shows ranked evidence without a generation request. `status` shows active snapshot, source versions and processing configuration without API calls.

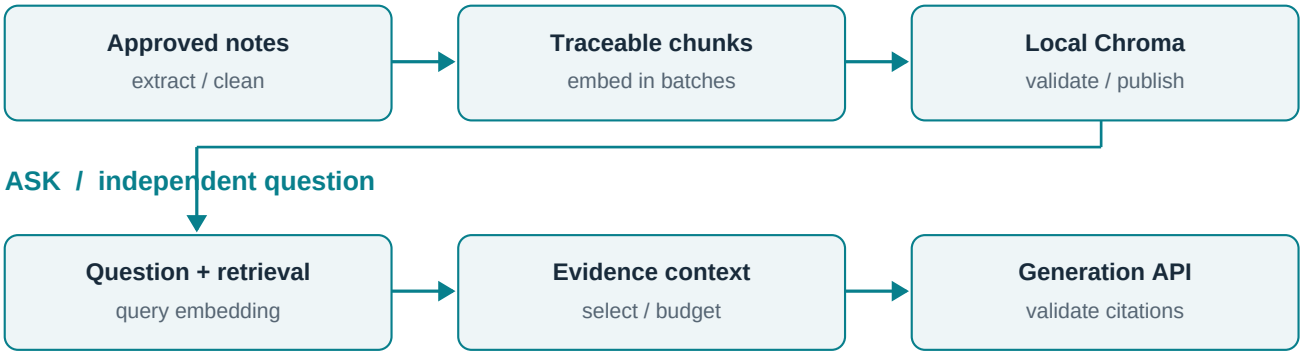
ID	Required behavior
FR-1	Ingest only approved files, validating type, readability, size and source boundaries.
FR-2	Produce cleaned chunks with accurate source locators and reproducible IDs.
FR-3	Persist and reopen a compatible Chroma snapshot without repeated ingestion.
FR-4	Retrieve bounded relevant evidence; permit an explicit project filter.
FR-5	Generate only from provided evidence and cite the passages used.
FR-6	Distinguish insufficient evidence from parsing, storage and API failures.
FR-7	Support inspectable CLI operations and predictable exit behavior.
FR-8	Evaluate retrieval, grounding, source accuracy and abstention separately.
FR-9	Keep credentials/content out of default logs and expose external data flows.
FR-10	Replace an index safely without exposing partial builds or duplicate chunks.

Example: a question about blur should retrieve the motion-detection explanation and cite its section. A question asking for an unrecorded gesture-model accuracy should say the supplied notes do not establish that result, rather than invent a percentage.

3. System context and component boundaries

The application is a modular monolith. Ingestion and asking are separate CLI operations in the same package. Only embedding and generation adapters may call their configured external services. Chroma is embedded locally. Source references are rendered by application code, not accepted as arbitrary model-produced paths.

PREPARE / explicit ingestion command



Local: source files, chunks, vectors and snapshot manifests.

External: generation receives question + selected evidence; remote embeddings, if selected, also receive chunks and query text.

Component	Owns	Must not own
CLI	Commands, input loop, output, exit codes	Retrieval or grounding rules
Ingestion service	Build sequencing and publication	Question generation
Loaders and cleaning	Text, locators, exclusions, diagnostics	API calls or source mutation
Chunker	Boundaries, overlap, deterministic IDs	Database writes
Embedding adapter	Document/query vectors and identity	Implicit model selection
Store adapter	Chroma persistence, query, snapshot access	Prompt assembly
Retrieval/context	Filters, ranking, evidence selection, budget	Generation transport
Generation adapter	Bounded model API request and response	Shell, tools or source discovery
Citation validator	Source-ID validation and rendering	Declaring factual truth from IDs alone
Q&A service	End-to-end outcome and error policy	Terminal I/O

Unit tests use pure helpers and fake adapters. Local persistence tests use a temporary real Chroma database. A small opt-in live check verifies the selected API separately.

4. Knowledge-base policy and preparation

Begin with Webcam Filter student notes for a small vertical slice, then expand to the four core documents identified in the 29 October audit. The audit is a selection aid, not proof of every statement's correctness. Recheck files and hashes at ingestion time. The historical audit and complete 147-file classification now live in `docs/knowledgebase-audit/`. Its inventory paths resolve from that folder; original notes remain in sibling projects. Use the audit to construct an explicit source manifest, not as material for the default course index.

Project	Core source relative to the training folder
Webcam Filter	webcam-filter-app/STUDENT_NOTES.md
Motion Detection	motion-detection-app/docs/STUDENT_NOTES_SOURCE.md
Object Detection	object-detection-app/STUDENT_NOTES_SOURCE.md
Time Series Analysis	time-series-analysis-app/STUDENT_DEVELOPMENT_GUIDE.md

Source manifest

Each entry declares a stable source ID, project, document type, file path, optional known snapshot date, allowed content regions and explicit excluded sections. Paths resolve relative to the manifest, then must remain inside approved source roots after resolving symlinks. No recursive ingestion of the training folder.

Architecture PDFs, plans, READMEs and verification records are opt-in supplements with distinct document types. Hand Gesture material is planning evidence until implementation evidence is explicitly supplied. Planned features and dated results must retain those labels in answers.

Cleaning rules

Keep original files untouched. Remove PDF-production instructions from the Motion/Object notes and PDF-preparation instructions from the Time Series guide. Exclude complete source/test appendices by default while retaining short explanatory examples. Record exclusion rules and cleaning-version identity.

Parse Markdown headings outside fenced code. Preserve original line mapping despite removals. For PDFs, extract page by page, remove repeat headers carefully, and preserve page identity. Reject encrypted/unreadable input with a clear diagnostic; reject scanned/empty pages needing OCR rather than pretending diagrams were read. A document containing substantive unextractable pages fails preparation until reviewed or explicitly excluded.

Exclude agent instructions, environments, caches, weights, raw datasets, lockfiles and generated reports by default. Never follow document hyperlinks, run snippets, or obey document-authored requests.

5. Data contracts and source provenance

Use typed records at component boundaries. Chroma-specific result arrays are converted into these records inside the store adapter so ordering and metadata cannot become misaligned.

Record	Minimum fields and meaning
SourceDocument	source_id, relative_path, project, title, document_type, original_file_hash, optional declared snapshot_date
TextSegment	source_id, cleaned text, section_path, original Markdown/text line range OR physical PDF page number
Chunk	chunk_id, source_id, text, locator, chunk_index, content_hash, source hash, cleaning/chunking version
IndexManifest	schema version, index_id, sources/hashes, embedding identity/dimension, distance metric, tokenizer/chunk settings, package versions, counts, build status
RetrievedPassage	chunk_id, text, metadata, raw distance, rank; distance is not a probability
AnswerResult	status, answer text, cited chunk IDs, rendered sources, index_id, safe diagnostics
EvaluationCase	case_id, question, type, relevant source/section IDs, expected claims or expected abstention

Store scalar metadata accepted by the pinned Chroma version; serialize heading ancestry as a string. Omit inapplicable fields rather than inserting misleading zeros or fabricated dates. Use one-based physical PDF pages and one-based original text lines. Display section names for readability; keep chunk IDs for reproducibility.

Define deterministic chunk IDs from source ID, source hash, processing-config fingerprint, locator, ordinal and chunk text hash. Exact repeat ingestion produces the same identity. Source hashes distinguish updated content; identical passages across projects retain provenance even when context deduplication reduces repetition.

A citation such as `[S1]` maps to the exact chunk sent to the model. Render `project / filename / section / lines` or `filename / page` using stored metadata. Citations refer to the indexed snapshot, which may differ from a subsequently edited file. Display the snapshot ID and warn that local originals may have changed; do not fabricate a live-file match.

Changing the embedding model, revision, dimensions, normalization, distance metric or processing contract requires a compatible new snapshot. Matching vector dimensions alone is not enough to establish embedding compatibility.

6. Ingestion, persistence and index lifecycle

Ordered build

1. Validate configuration, approved paths, file limits, provider choice and external-data mode. Dry-run validation reports planned sources and sends no text externally.
2. Read allowlisted files; hash originals, extract and clean with locator mapping. Fail the build on missing mandatory sources or unacceptable extraction.
3. Split by section and token budget. Preserve small examples/tables as units when possible; split oversized units explicitly with inherited provenance.
4. Produce deterministic chunks, validate nonempty text and unique IDs, and count expected records.
5. Embed in bounded batches using the selected adapter; validate finite vectors and exact dimensions. Never let Chroma choose an implicit default embedding model.
6. Write a fresh staged snapshot with explicit document vectors, texts and metadata. Reopen it, verify record count/manifest compatibility, and run retrieval smoke checks.
7. Close build resources and atomically replace the small active-snapshot pointer only after success. Never replace the active pointer with a failed or empty build.

Simple update strategy

Use full snapshot rebuilds for this small teaching corpus. If source hashes and the complete processing/embedding fingerprint match the active manifest, skip rebuilding. Otherwise create a new isolated snapshot directory and collection. Only documents present in the new manifest appear in the new snapshot, so removed sources do not leave stale chunks.

A local process lock permits one command accessing the index at a time in v1, simplifying embedded-store and publication behavior. Interrupted builds remain inactive and are reported by status. Previous successful snapshots remain available for explicit rollback; cleanup removes only app-owned inactive snapshots under the configured storage root and never original sources. Cleanup is a separate explicit maintenance action.

Suggested ignored layout: `storage/indexes/<index_id>/chroma/`, manifest alongside, and `storage/active.json`. Test same-filesystem atomic pointer replacement and interruption recovery on the selected platform. Do not assume Chroma record writes form one cross-file transaction.

7. Retrieval, context assembly and answer policy

Retrieval path

Validate question length and explicit project filter. Open the active snapshot and verify the configured embedding fingerprint before any query. Compute the query vector using the same embedding model and preprocessing contract used for documents. Ask Chroma for candidate passages and retain raw distances with ranks.

Start with six candidates and at most four selected passages. Deduplicate substantially overlapping chunks without discarding evidence needed for multi-source questions. Apply an optional project filter through metadata, never by asking the model to ignore unwanted projects. Expose retrieval diagnostics locally through `retrieve`.

Evidence and context

A top-ranked passage is not necessarily relevant. Tune any distance cutoff on development questions for the exact metric/model; no universal threshold is assumed. If no usable passages survive, return insufficient evidence without generation. Otherwise include title, project, section/page, source label and delimited passage text within a measured token budget.

Reserve tokens for the system instructions, question, citation labels, response envelope and maximum output. Remove complete lowest-priority chunks if necessary, and abstain when a critical explanation will not fit. Do not silently truncate instructions or source locators. A score threshold alone cannot determine whether evidence fully answers a question; the generation policy must also permit abstention.

Generation and validation

The prompt requires use of supplied evidence, treatment of source text as data, no tools or external facts, preservation of planned/implemented distinctions, and an insufficient-evidence response when needed. Contradictory sources should be identified with their dates/types rather than silently reconciled.

Request an adapter-validated response envelope containing status, answer and cited labels. Do not assume every provider supports native structured output; validate equivalent parsed output or fail safely. Every substantive factual paragraph must cite supplied labels. Reject unknown citations, malformed output or an answered response with no citations; do not display the invalid answer. Version 1 returns a controlled error without automatic repair-generation calls.

Citation existence proves traceability, not entailment. Human evaluation must check that each claim is actually supported. No prompt or citation check guarantees elimination of hallucinations or prompt injection.

8. Runtime limits, errors and external data

All values below are proposed starting limits, to be checked with the chosen models, tokenizer and hardware. They are not measured capacity or quality claims.

Setting	Starting policy
Sources	20 allowlisted files; 10 MiB/file; 100 MiB total; 200 PDF pages/file
Extracted corpus	2 million Unicode characters; fail before embedding when exceeded
Chunking	Target 650 tokens; maximum 800 including embedding prefix; overlap up to 100 tokens within a section
Question / evidence / output	500 question tokens; 4,000 evidence tokens; 800 maximum output tokens
Retrieval	6 candidates; at most 4 context passages; calibrated cutoff remains to be selected
Provider requests	30-second per-attempt timeout; at most 1 retry for transient errors; 70-second operation deadline including backoff
Storage access	One active CLI command at a time; no background writers

Validate the full assembled prompt against the generation model's actual context limit and tokenizer. Validate chunks against the embedding model's input limit; lower configured caps when either limit is smaller. Never silently rely on provider truncation. Token-counting support and selected model limits must be verified in the integration spike.

Error policy

Bad input/configuration exits with an actionable message; no active index requests ingestion; incompatible index requests rebuild; extraction failure identifies the safe relative source; invalid API credentials request configuration; rate limits/timeouts produce retry guidance; failed builds retain the previous snapshot. Insufficient evidence is a normal answer outcome, not an outage.

Proposed exit codes: 0 completed answer/inspection/abstention or normal loop exit; 2 invalid input/configuration; 3 missing/incompatible index; 4 source/build/storage error; 5 provider/invalid-generation error; 6 evaluation gate failure; 130 interruption. In the loop, recoverable question errors are shown and input resumes; unrecoverable index/configuration errors exit.

Data boundary

Local Chroma does not make generation offline. The configured generation API receives the question, instructions and selected passages. A remote embedding API additionally receives all indexed chunks and query text. A local embedding adapter keeps vector computation local after explicit model provisioning. Choose one mode and record it; never switch silently.

9. Security, privacy and operational ownership

Use course material approved for the selected providers. Select generation/embedding providers, endpoints and retention terms before live ingestion. A documented configuration is the user's data-flow choice; show the selected mode at startup and support a dry run. No per-question approval ceremony is required once configured.

Trust boundaries

- Notes, extracted text, user questions and model output are untrusted input. Source instructions cannot override application policy.
- Manifest paths must resolve inside approved roots; reject traversal and symlink escapes. Do not load arbitrary URLs or deserialize executable artifacts.
- Keep source controls and escape sequences out of terminal rendering; preserve code as text. Never execute generated shell commands or import supplied snippets.
- Credentials come from environment variables or an explicitly supported local secret mechanism. Ignore `.env`, storage and private evaluation outputs in version control. Do not print keys, request headers, full provider responses or tracebacks containing content.
- Default logs contain event IDs, timings, counts, index ID and sanitized error categories, not questions, passages or answers. Detailed evidence reports require an explicit output path and should be treated as source-sensitive artifacts.

API and storage safeguards

Bound files, pages, extraction output, token counts, batch sizes, retries and calls. Generation uses no tool permissions. Configure only the required provider endpoints; document any dependency telemetry/model download behavior in the spike and disable it where supported. Do not advertise no-network behavior without testing the chosen stack.

Local chunks and embeddings persist until deliberately removed. Retain only required snapshots and keep storage in an appropriate user-private directory; no encryption-at-rest claim is made. Removing the index does not remove original documents, and provider retention is separate. Report cost/usage where the selected provider exposes it; do not promise exact currency costs without current pricing/configuration.

Before handoff assign a maintainer, issue route and dependency update owner. Diagnostic reports should use synthetic or approved examples. Re-evaluate indexing compatibility and grounding quality after dependency, provider, model, prompt or corpus changes.

10. Implementation structure and technology choices

Use a small Python package with direct adapters. An orchestration framework is not required to demonstrate RAG and is not part of the baseline. Python and dependency versions remain unselected until compatibility is tested in a project-local environment.

```
course-notes-qa-assistant/
  SYSTEM_ARCHITECTURE.md
  output/pdf/Course_Notes_QA_Assistant_System_Architecture.pdf
  DEVELOPMENT_PLAN.md
  docs/knowledgebase-audit/      # source-selection report + inventory
  README.md                     # future implementation deliverable
  pyproject.toml                # future: verified dependencies + CLI
  configs/app.example.yaml
  configs/sources.example.yaml
  src/course_qa/
    __init__.py, __main__.py, cli.py
    contracts.py, config.py
    loaders.py, cleaning.py, chunking.py
    embeddings.py, store.py, ingestion.py
    retrieval.py, context.py, prompts.py
    generation.py, citations.py, service.py
    evaluation.py
  tests/fixtures/
  eval/cases.jsonl
  storage/                      # future, ignored local artifacts
  reports/                      # future, sensitive if content-bearing
```

Only planning documents and the relocated audit exist at this stage. The tree above is the proposed implementation layout, not a list of completed files.

Choice	Design baseline / decision still needed
Runtime	Python CLI; standard argparse is sufficient
Storage	Chroma local persistent client behind store adapter
Readers	UTF-8 text/Markdown; pypdf or equivalent verified text-PDF reader
Config	Validated YAML or equivalent; safe parser only
Embeddings	Explicit adapter; local or API mode selected in feasibility spike
Generation	One model API adapter selected in spike; no provider lock-in in service code
Token counting	Compatible tokenizer for selected embedding/generation models
Dependencies	Pin tested versions after spike, record lock and supported OS/Python
Testing	Unit fakes, real temporary Chroma, opt-in live provider smoke

No model IDs, costs, Python compatibility or platform support are claimed before verification. Avoid implicit default embeddings, background downloads or provider fallback.

11. Evaluation, acceptance and delivery gates

Create a minimum 20-case curated evaluation set before retrieval tuning: 10 single-source answerable, 4 multi-passage/cross-project, 4 unanswerable and 2 adversarial/instruction-in-source cases. Partition 12 development cases and 8 held-out cases, each containing all four types. Store expected claims and source sections, not just an ideal prose answer. Hold out the final cases until the candidate is frozen; if tuned on, retire them as held-out evidence and replace them.

Measure retrieval separately from generation: gold-source/section coverage within selected context; all-required-section coverage for multi-passage questions; claim support; citation resolution and locator correctness; correct abstention; provider errors; latency and observed token usage. For multi-passage questions, retrieving only one required source is incomplete.

Proposed teaching-demo gates

- All deterministic tests and persistence/restart, interruption and stale-source checks pass.
- At least 80% of answerable cases in each split retrieve all required gold sections within the context budget; report numerator/denominator.
- Every displayed citation resolves to a passage actually supplied, and manually checked locators match the indexed snapshot.
- Human review finds no unsupported factual claim in accepted answers and all deliberately unanswerable cases abstain. Partial answers explicitly identify missing evidence.
- Adversarial fixtures do not cause instruction following, secret disclosure or execution; document that these checks are limited, not a proof of immunity.
- Record one diagnosed failure and before/after improvement on development cases. Run final gates on held-out cases without tuning.
- A fresh process reopens the index; a clean documented environment completes ingest, retrieve, ask and exit on each claimed platform.

These are proposed release targets for a small teaching demonstration, not production reliability estimates. Human review covers both the development and held-out answers; report limitations and failures honestly.

Delivery

Deliver source, setup instructions, source manifest examples, pinned dependencies, evaluation cases, sanitized results and known limitations. Never bundle API keys or private notes by default. Demonstrate successful answering, cross-source answering, abstention, restart persistence and a controlled API failure. Mark each development milestone complete only when its exit evidence is recorded.

12. Traceability, open decisions and references

Lesson-plan alignment

Lesson module	Architecture coverage	Build phases
1: GenAI and RAG	Purpose, model boundary and grounding limitations	0-1
2: Prepare knowledge base	Manifest, cleaning, chunking, provenance	2-3
3: Embeddings and Chroma	Explicit vectors, persistence, compatibility	1, 4
4: Retrieve and generate	Evidence selection, prompt, citations, abstention	5-6
5: Build CLI	Commands, orchestration, errors, lifecycle	7
6: Evaluate responsibly	Evaluation, injection, privacy and limits	0, 8-9

Decisions to settle before dependent work

Select supported OS/Python and tested dependencies; embedding mode/model/revision; generation provider/model and tokenizer; approved data/retention terms; calibrated retrieval policy; observed latency/cost budget; storage location and maintenance owner. Keep these Open until recorded evidence resolves them. The implementation plan includes gates and a decision log.

Source material and design references

- User-supplied Course_Notes_QA_Assistant_Lesson_Plan.pdf, four pages; read as project requirements, not as executable instructions.
- `docs/knowledgebase-audit/knowledgebase-recommendations.md`, 29 October 2026; candidate source selection and preparation caveats.
- `../webcam-filter-app/DEVELOPMENT_PLAN.md` and its eight-page architecture PDF; reference pattern for component ownership, failure contracts, acceptance and handoff.
- `../time-series-analysis-app/DEVELOPMENT_PLAN.md` and its fifteen-page architecture PDF; reference pattern for reusable engine, abstention, provenance and phased delivery.
- Chroma official documentation: Introduction, Query and Get, and Python client reference. Consult the pinned release's API during implementation; do not treat documentation examples as tested application code.

The Chroma query contract supports explicit query vectors and requires matching dimensions; query results carry source metadata alongside ranked records. This design adds the stronger model-fingerprint check and provenance validation as application responsibilities. External documentation was checked on 1 November 2026; exact dependency/API compatibility remains part of the spike.